Bayes, MAP, MLE (W1)

P(h|D) = P(D|h)P(h)/P(D) post = likelihood × prior / evidence

Total prob: P(D) = Σh P(D|h)P(h); P(S)=P(S|M)P(M)+P(S|¬M)(1−P(M))

MAP: argmax_h P(D|h)P(h) (drop P(D)). MLE: argmax_h P(D|h)

MLE = MAP if prior uniform. iid: P(D|h) = Π p(xi|h)

Classify: Ck = argmax p(x|Ck)P(Ck); posterior = score/Σ scores

Check: P(S|M)=0.5, P(M)=1/50000, P(S)=1/20: P(M|S)=0.0002

Gaussian fit + Gaussian NB (Q2)

μ = (1/n)Σxi σ^2 = (1/n)Σ(xi−μ)^2 (MLE: divide by n)

State "MLE, /n". Unbiased /(n−1) only if asked (= σ^2 × n/(n−1))

1-D: p(x) = 1/√(2πσ^2) × e^(−(x−μ)^2/(2σ^2))

Recipe per class k: prior P(Ck) = nk/n μk = mean of each feature over class-k points only table: x, x−μ, (x−μ)^2, sum, ÷ nk (do per feature) Σk = diag($\sigma1^2$, $\sigma2^2$), off-diagonals 0 (cond. independence)

Classify x: score_k = P(Ck) × Πj N(xj | μkj, σkj^2); max wins

Posterior = score_k/Σ scores. Tiny numbers: compare ln scores

Full Gaussian: cov(x,y) = (1/n)Σ(x−μx)(y−μy); naive sets it 0

Params: naive K×2d + (K−1); full K(d + d(d+1)/2) + (K−1)

Check: x = −1,1,2,2,4: μ=1.6, Σ(x−μ)^2=13.2, σ^2=13.2/5=2.64

Discrete NB + Laplace

P(xj=v|Ck) = count(v and Ck)/count(Ck)

C = argmax_k P(Ck) Πj P(xj|Ck)

Tennis (sunny,cool,high,strong): yes 0.0053 < no 0.0206: No

Laplace: P(w|C) = (count(w,C)+1)/(total words in C + |V|)

Why: one zero count makes the whole product 0

Trees: entropy, IG, Gini, gain ratio (W4)

H(S) = −Σ pi log2 pi; 2 classes: max 1 at 50/50, pure = 0

H(S|A) = Σv (|Sv|/|S|) H(Sv); IG = H(S) − H(S|A); pick max

Gini = 1 − Σpi^2; GiniGain = Gini(S) − Σv (|Sv|/|S|)Gini(Sv)

SplitInfo = −Σv (|Sv|/|S|) log2(|Sv|/|S|); GR = IG/SplitInfo

Calc: log2(x) = ln(x)/ln(2) = ln x ÷ 0.6931; 0×log2(0) = 0

Check: H(9+,5−)=0.940, H(2,3)=0.971, Gini(9,5)=0.459

Tennis: IG Outlook 0.247 > Humidity 0.152; GR=0.247/1.577=0.156

Continuous: sort, threshold θ = midpoint where class changes

Confusion matrix, ROC, AUC (Q6)

Rows actual, cols predicted: [TP FN; FP TN]

Acc = (TP+TN)/all Prec = TP/(TP+FP) Rec = TPR = TP/(TP+FN)

FPR = FP/(FP+TN) Specificity = TN/(FP+TN) = 1 − FPR

F1 = 2PR/(P+R) Fβ = (1+β^2)PR/(β^{2}P + R)

ROC: x = FPR, y = TPR per threshold; add (0,0),(1,1); diagonal

AUC = area: 0.5 random, 1 perfect. Trapezoid Σ(x2−x1)(y1+y2)/2

Points below diagonal (TPR < FPR): worse than random

Fix: invert classifier (swap +/− decisions, or use 1 − score) (FPR,TPR) becomes (1−FPR, 1−TPR); new AUC = 1 − old AUC

Imbalanced: accuracy misleading, prefer precision-recall curve

Q6 t=0.5: TP5 FN15 FP30 TN50: TPR=5/20=0.25, FPR=30/80=0.375

Linear regression (W2)

h(x) = w0 + w1x1 + ... = w^{T}x (x0=1); MSE J = (1/n)Σ(h(xi) −yi)^2

∂J/∂w0 = (2/n)Σ(h(xi)−yi); ∂J/∂w1 = (2/n)Σ xi(h(xi)−yi)

GD: wj ← wj − α ∂J/∂wj, all j updated simultaneously

Normal eq: w = (X^{T}X)$^{-1}$X^{T}y

R^2 = 1 − SSE/SST; SSE = Σ(yi−ŷi)^2, SST = Σ(yi−ȳ)^2

Ridge J + α‖w‖$_2^2$ = αΣw^2; Lasso J + α‖w‖$_1$ = αΣ|w|

E[error] = Bias^2 + Variance + σ^2 (irreducible noise)

Logistic, softmax, Q3 d+1 (W3)

σ(z) = 1/(1+e^(−z)); σ(0)=0.5; σ(−z) = 1−σ(z); σ' = σ(1−σ)

p(1|x) = σ(w^{T}x + b); log-odds ln(p/(1−p)) = w^{T}x + b (linear)

Boundary w^{T}x + b = 0 (p = 0.5); 1-D: x* = −b/w

Log loss y∈{0,1}: −Σ[y ln ŷ + (1−y) ln(1−ŷ)]; ∂/∂z = ŷ − y

y∈{±1}: L = Σ ln(1 + e^(−y w^{T}x))

Softmax p(k|x) = e^(zk)/Σj e^(zj), zk = wk^{T}x + βk: K(d+1) params

Multiclass CE = −ln p(true class)

Q3: K=2, divide top and bottom by e^(z1): p(1|x) = 1/(1 + e^(z2−z1)) = 1/(1 + e^(−(w1−w2)^{T}x − (β1−β2))) = σ(w^{T}x + β), w = w1−w2, β = β1−β2: only differences, d+1 Add same (c, b) to both classes: probs unchanged, redundant General K: (K−1)(d+1) free params; K=2 gives d+1

Check: σ(2) = 0.881; −ln(0.8) = 0.223

Bootstrap, bagging (Q4)

n draws with replacement. P(not picked in 1 draw) = 1 − 1/n

P(never picked in n draws) = (1 − 1/n)^n, tends to 1/e = 0.368

P(selected) = 1 − (1 − 1/n)^n, tends to 1 − 1/e = 0.632

n=10: 0.651, n=100: 0.634, n=1000: 0.632

OOB 36.8% = free validation set (oob_score)

Average of B iid models: Var = σ^2/B

AdaBoost (course, labels ±1)

err_t = Σ w_i [h_t(x_i) ≠ y_i] / Σ w_i

α_t = ½ ln((1 − err_t)/err_t); err 1/3: α = ½ ln 2 = 0.347

w_i ← w_i e^(−α_t y_i h_t(x_i)): wrong × e^α, right × e^(−α)

Then renormalise so Σ w_i = 1

Equal start, k wrong of n: wrong = 1/(2k), right = 1/(2(n−k))

Final H(x) = sign(Σt α_t h_t(x))

Notebook/sklearn variant α = ln((1−ε)/ε): say which you use

Perceptron, neuron (W5)

ŷ = 1 if Σ wjxj > 0 else −1 (x0 = 1 is bias); labels ±1

Step perceptron: w ← w + η(y − ŷ)x; wrong gives y − ŷ = ±2

Slide SGD eg (ŷ = w.x, J = (y−ŷ)^2): w ← w + 2η(y − ŷ)x

η = 0.5: w ← w + (y−ŷ)x; w (0,0,0) to (1,1,1) to (−2,4,−5)

Exam: write the rule you use before applying it

Sigmoid + BCE step: w ← w − η(ŷ − y)x

MLP params m(d+1) + k(m+1); dense layer n_in×n_out + n_out

CNN sizes and params (Q5)

O = (N + 2P − F)/S + 1; P=0, S=1: O = N − F + 1

Pool 2×2 stride 2: O = N/2, depth unchanged, 0 params

Conv params = K × (F×F×C_in + 1); C_in = depth of input

Output O×O×K. Check: 32×32×3, 6 of 5×5: 28×28×6, 456 params

Swap question: write both configs as formulas, subtract, sign

Q5 pool1: (p−s+1)/2, s1 to smaller s2: INCREASE pool2: (p−s1−2s2+3)/4; swapped − correct = (s2−s1)/4: DECREASE conv1 f1(3s^2+1): DECREASE; conv2 f2(f1s^2+1): INCREASE total diff = f1($s1^2$−$s2^2$)(f2−3) > 0: INCREASE (big kernel, f1 ch)

Attention (W7)

Attention(Q,K,V) = softmax(QK^T/√dk) V, softmax per row

score q.k/√dk, weights α sum to 1, output Σ αi vi

RNN params dn + n^2 + n; LSTM 4(n(n+d) + n) (input d, hidden n)

2 keys: α1 = 1/(1 + e^(s2−s1)). Causal mask: n(n+1)/2 of n^2

k-means (Q7) + distances

Init k centres. Repeat: 1) assign each point to nearest centre

2) update: centre = mean of its points (x̄, ȳ)

Converged when assignments unchanged (centres stop moving)

Table: point | d to c1 | d to c2 | cluster; 3 dp for near-ties

SSE = Σ‖xi − μ(ci)‖^2 never increases; only a local optimum

Euclid d = √((x1−x2)^2 + (y1−y2)^2); (2,8) to (5,5) = √18 = 4.243

Cosine = (a.b)/(|a||b|): (1,2),(2,1) gives 4/5 = 0.8

Silhouette s = (b−a)/max(a,b); k-means++: P(x) ∝ D(x)^2

Calculator keys (fx-96SG PLUS / 570ES / 991ES)

Setup: SHIFT MODE 1 then 2 (LineO); SHIFT MODE 8 then 2 (Norm 2)

log2(x) = log(x) ÷ log(2); e^x = SHIFT ln; STO = SHIFT RCL

STAT: MODE 2 (96SG) or MODE 3 (ES), 1 (1-VAR), x =, ..., AC

SHIFT 1 4: 1 n, 2 mean, 3 σx (÷n, USE), 4 sx (÷n−1)

Variance = σx^2 ; clear data in editor SHIFT 1 3 2 (Del-A)

S ⇔ D for decimal. Bracket: e^(−(x−μ)^2/(2σ^2)), 1/(1+e^(−z))

Verified mini answers (check your method)

GNB Q2 model, x=(2,4): coef 1/√(2π×2.64) = 0.2455 c0: e^(−0.16/5.28)=0.9702, dens 0.2382, ×2 feats: 0.0567 c1: e^(−1.96/5.28)=0.6899, dens 0.1694, ×2 feats: 0.0287 P(c0|x) = 0.0567/(0.0567+0.0287) = 0.6641: class 0

Bayes P(D)=.01,P(+|D)=.9,P(+|¬D)=.05: .009/.0585 = 0.1538

IG S[29+,35−] H=0.9937; A1 [21,5][8,30]: H|A1=0.7278 IG(A1)=0.2659 > IG(A2 [18,33][11,2])=0.1214: split A1

Logit w=(0.8,−0.5), b=−1, x=(3,2): z=0.4, p=0.5987

AdaBoost 10 ex w=0.1, 3 wrong: ε=0.3, α=0.4236 e^α=1.5275, e^−α=0.6547, Z=0.9165: wrong 1/6, right 1/14

Backprop slide: x=(1,1,2), y=2, z=(1,.5,.5), ŷ=1, J=1 ∂J/∂W2 = 2(ŷ−y)z = (−2,−1,−1); hidden δ = −2×1×0.25 ∂J/∂W1 rows (−.5,−.5,−1); η=0.1: ŷ=1.4638, J=0.2875

Attn q=(1,0,2,1): q.k = 1,3,4; ÷√4 = 0.5,1.5,2 softmax (0.1220,0.3315,0.5465); v=(10,2,6): out 5.1618

Q5 p=64,s1=9,s2=5,f1=15,f2=27: pool1 28 to 30, pool2 12 to 11; params 13812 to 33972 (+20160 = f1(81−25)(24))

Q7 it1 centres (4.6,8.2),(9.4,6.8); it2 (5,9.6),(9,5.4) C and H switch in it2, so NOT converged; stable from it4

Answer templates

- Explain: claim, reason (why), formula or tiny example, so-what
- Compare: what each does, pro, con, when to use (mini table)
- Calc: formula, substitute numbers, result (3 dp), interpret
- Trend/sign: write both cases as formulas, subtract, sign
- Define symbols, state convention (MLE /n, log2, labels ±1)
- ~1.7 min per mark. Method marks even if a number slips

Validation, CV, leakage (Q1)

- Train: fit params. Val: choose model/hyperparams. Test: once
- Validation split comes FROM the TRAINING set; test untouched
- Tuning on test = leakage: test score optimistically biased
- Training error is optimistic, so cannot choose models with it
- Hold-out: 1 split, cheap, high variance, val pts never trained
- k-fold CV: k folds, train on k−1, validate on 1, rotate, average
- CV: each point validated once, lower variance, k× cost
- Stratified k-fold keeps class ratios; time series: past to future
- Leakage: fit scaler/feature selection on train only
- Tune by grid search / random search, scored with CV

Bias-variance, under/overfitting

- Bias: error from too-simple assumptions (underfit)
- Variance: sensitivity to the training sample (overfit)
- Underfit: train and val error both high: more complex model, more features, less regularisation
- Overfit: train low, val high: more data, simpler model, regularisation, pruning, dropout, early stopping, ensembles
- More complexity: bias down, variance up; pick val-error min
- Occam: prefer the simplest model that fits the data

Regularisation L1 vs L2

- Loss + $\alpha\|w\|$: penalise big weights; larger α = simpler, more bias
- L2 Ridge $\alpha\Sigma w^2$: shrinks all weights smoothly, rarely exactly 0
- L1 Lasso $\alpha\Sigma|w|$: sets some w exactly 0 = feature selection
- Elastic net = both. Scale features first
- NN form $C \times J + \|w\|$: large C = weaker regularisation

Generative vs discriminative

- Generative: models p(x|C)P(C), can make new x: NB, GDA, GAN
- Discriminative: models p(C|x)/boundary: logistic, trees, NN
- Gen: little data ok, strong assumptions; Disc: more accurate with lots of data, fewer assumptions about x
- NB: features cond. independent given class; seldom true, works

Decision trees

- ID3: greedy top-down, split on max IG attribute, recurse
- Pros: interpretable, no scaling, numeric + categorical, non-parametric, multi-class, feature selection
- Cons: overfits (high variance), unstable, loses info on cont.
- Pre-prune: max depth, min samples split/leaf
- Post-prune: reduced-error pruning on validation set
- IG favours many-valued attrs (ID, date): use gain ratio
- Impure leaf: majority vote; regression: variance reduction

Ensembles

- Bagging: bootstrap samples, parallel, equal vote: cuts VARIANCE
- RF: bagged trees + random feature subset per split: decorrelated
- Boosting: sequential, upweight errors, weighted vote: cuts BIAS AdaBoost; gradient boosting fits residuals; can overfit noise
- Stacking: meta-model learns to combine base model predictions (train on out-of-fold preds); blending = on a held-out set
- Needs base models better than chance AND diverse errors

Neural networks (W5)

- Neuron $a = f(\Sigma wx + b)$. No non-linearity: layers collapse to one linear map, so cannot learn XOR / curved boundaries
- $\sigma \in (0,1)$, $\sigma' = \sigma(1-\sigma) \le 0.25$ (max at 0)
- $\tanh \in (-1,1)$, $\tanh' = 1 - \tanh^2$, $\tanh(z) = 2\sigma(2z) - 1$
- ReLU = max(0,z), ReLU' = 1 if z>0 else 0; leaky: az for z<0
- Output: sigmoid (binary), softmax (K), identity (regression)
- Universal approx: 1 hidden layer, enough units, any function
- Backprop: forward, loss, chain rule backward, GD update $\partial J/\partial w = \partial J/\partial \hat{y} \times \partial \hat{y}/\partial v \times \partial v/\partial w$; sigmoid/softmax+CE: $\delta = \hat{y} - y$ hidden $\delta q = \delta\, wq\, zq(1-zq)$; $\partial J/\partial W(l) = \delta(l)\, z(l-1)^T$ $W \leftarrow W - \eta\, \partial J/\partial W$
- Vanishing grad: many $\sigma' \le 0.25$ multiplied, early layers stall. Fix: ReLU, Xavier/He init, normalisation, skip conn, LSTM
- Exploding grad: gradient clipping
- Dropout: drop units w.p. 1−α in training, scale by α at test; ≈ cheap ensemble of 2^units sub-networks
- Early stopping: stop at minimum validation error
- Batch norm: normalise layer inputs per batch, faster, stabler
- Epoch = 1 pass over data; mini-batch between batch GD and SGD

CNN (W6)

- Local connectivity: neuron sees a small patch (receptive field)
- Weight sharing: same filter everywhere, far fewer params
- Translation invariance: sharing + pooling, pattern anywhere
- Pooling: downsample, no params, robust to small shifts
- Receptive field grows with depth: two 3×3 see 5×5, fewer params
- Layers learn edges, then textures, parts, objects
- ResNet: y = ReLU(F(x) + x); skip lets gradient flow, very deep
- FC on raw image: huge param count, ignores spatial structure

RNN, LSTM, attention, transformer (W7)

- RNN: $h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$, same W every step, BPTT
- Long sequences: gradient ~ W^T: vanishes (<1) / explodes (>1)
- LSTM: cell state C = conveyor belt; forget, input, output gates ($\sigma \in [0,1]$); additive path keeps long-range gradients
- Attention: weighted average of values, weights from query.key
- vs RNN: any 2 positions linked directly, parallel; cost $O(n^2)$
- Multi-head: several Q/K/V projections, different relations, concat then × W^O
- Positional encoding (sin/cos) added: attention is order-blind
- Masked self-attn (decoder): token sees only earlier tokens
- Add & Norm: residual + layer norm. Cross-attn: dec Q, enc K,V
- BERT: encoder-only, bidirectional, masked words: understanding
- GPT: decoder-only, causal mask, next-token: generation

Generative AI (W8)

- GAN: generator (noise to fake) vs discriminator (real/fake); min-max game, trained in alternation
- LLM: 1) pre-train next word, self-supervised, huge text 2) instruction fine-tuning (SFT) 3) RLHF
- RLHF: reward model from human rankings, $-\log \sigma(r_w - r_l)$; PPO maximises reward − KL penalty to base model
- Good pre-trained init is why fine-tuning works
- Distillation: small student mimics big teacher outputs
- Diffusion: forward adds noise; train noise predictor; generate by iterative denoising from pure noise
- Stable Diffusion: text encoder + latent denoiser + decoder
- VAE: $z = \mu + \sigma\varepsilon$. Eval: FID (lower better), CLIP score
- Text sampling: greedy/beam repeats; need randomness

Unsupervised (W9)

- k-means cons: choose k, init-sensitive (local opt), assumes round similar-size clusters, outliers, needs scaling
- Fixes: several restarts, k-means++ init, GMM for ellipses
- Choose k: elbow (SSE vs k, bend); silhouette (max avg s)
- Agglomerative: merge closest pair, dendrogram, cut for k
- Linkage: single = min dist, complete = max, average = mean
- DBSCAN: density (eps, minPts), any shape, flags noise
- PCA: centre, $\Sigma = (1/n)\ddot{X}^T\ddot{X}$, eigenvectors by λ, keep top k var explained = $(\lambda 1 + \ldots + \lambda k)/\Sigma\lambda$; linear, max variance
- Embeddings: dense vectors, similar meaning = close (cosine)
- word2vec: CBOW context to word; skip-gram word to context
- tf-idf: idf = log(|D|/df); LSA = truncated SVD of doc-word X

Ready-to-write model answers

- CNN vs FNN: local connectivity + weight sharing = far fewer params; translation invariance; hierarchy edges to objects
- Bagging vs boosting: parallel vs sequential; cuts variance vs bias; equal vs α-weighted vote; boosting sensitive to noise
- Attention vs RNN: parallel, direct long-range links, no vanishing through time; cost $O(n^2)$ in sequence length
- Overfit fixes: more data, L1/L2, simpler model, dropout, early stopping, augmentation, pruning, tune with CV
- Imbalanced: accuracy misleads; use P/R/F1/PR-AUC, stratified CV, class weights, resampling, move the threshold
- Validation not test: test must stay unseen; tuning on test leaks it, so the reported score is too optimistic